Operator Decomposition for Stochastic Multi-Agent Path Finding

An Empirical Comparison of Baseline RTDP and OD-RTDP

Tal Rays and Eli Ben-Shimol
Collaboration in AI, Ben-Gurion University
July 2026

Abstract

Stochastic multi-agent path finding requires a policy over joint states rather than a single fixed plan, but direct planning becomes increasingly expensive as the number of coordinated agents grows. This study compares a baseline Real-Time Dynamic Programming (RTDP) planner that evaluates complete joint actions with an Operator-Decomposition RTDP (OD-RTDP) planner that constructs each joint decision sequentially. Both planners operate in the same fully cooperative MMDP and differ only in the representation of action selection. The open-grid results reveal a clear computational crossover, while the full benchmark indicates a broader scale-dependent trade-off: direct joint-action evaluation is preferable for very small teams, whereas OD-RTDP becomes increasingly advantageous as the number of agents grows. In the tested conditions, this change is reflected in substantially lower planning time and peak memory at the larger team sizes. The advantage does not extend to every aspect of the problem, because OD introduces prefix-state overhead, does not remove joint-state growth, and does not consistently improve policy-evaluation outcomes. The study therefore supports operator decomposition as a more scalable planning representation when joint-action branching is the dominant computational bottleneck.

1. Introduction

Multi-Agent Path Finding (MAPF) concerns the coordinated movement of several agents from assigned start positions to assigned destinations while preventing conflicts. The problem appears in systems such as warehouse automation, mobile robotics, and shared transportation environments, where several decision makers operate in the same physical space.

In deterministic formulations, an intended move is assumed to occur as planned, allowing a solution to be represented as a fixed sequence of joint actions. Stochastic execution changes this requirement. A movement may fail, an agent may remain in place, and the resulting joint configuration may differ from the one expected by the original plan. The solution must therefore be a policy that selects an action for each relevant joint state that can arise during execution.

This study models the team as a fully cooperative, centralized Multi-Agent Markov Decision Process (MMDP). All agents share one team-cost objective, the planner observes the complete joint state, and a single joint policy determines one local action for each agent. This formulation captures collision constraints, but it creates two related forms of combinatorial growth: the number of possible team configurations increases with combinations of agent positions, and the number of complete joint actions increases with combinations of local actions. This rapid growth is an instance of the curse of dimensionality in multi-agent decision making.

Operator decomposition changes how a joint decision is represented computationally. Instead of comparing every complete action combination at once, the planner constructs the joint action through a sequence of local choices. This serialization reduces the branching factor at each intermediate planning step, but it also introduces action-prefix states and greater decision depth. OD changes only the planner's internal construction of the decision; once the prefix is complete, the selected local actions are executed simultaneously in the environment.

The study examines how this trade-off affects the computational performance of Real-Time Dynamic Programming as the number of coordinated agents and the structure of the environment vary. The comparison focuses on planning convergence, planning time, peak memory consumption, completion of the full experimental condition, and observed policy behavior under stochastic execution.

The main contribution is an empirical characterization of the conditions under which operator decomposition becomes computationally beneficial in stochastic multi-agent planning. The study distinguishes a representation-level advantage from a universal performance claim: decomposition can reduce the burden of complete joint-action evaluation, but it does not eliminate growth in the physical joint-state space and does not guarantee superior policy behavior in every environment.

2. Background

2.1 Cooperative MMDPs and Stochastic Shortest Paths

A Multi-Agent Markov Decision Process extends a Markov Decision Process to settings in which several agents act within one environment. In a fully cooperative MMDP, all agents share the same objective, so the team can be represented as one collective decision maker. The joint state describes the complete team configuration, the joint action contains one

local action for each agent, and the transition model specifies a probability distribution over successor joint states. Under the fully observable and centralized assumption used here, the planner has access to the complete joint state and computes a stationary joint policy pi: S -> A, which assigns a joint action to every relevant state.

A Stochastic Shortest Path (SSP) problem is a cost-minimization decision process whose objective is to reach a designated goal set while minimizing expected cumulative cost. In an undiscounted SSP, future costs are not reduced by a discount factor; the objective is the expected total cost accumulated until a goal is reached. The optimal value function represents the minimum expected remaining cost from each state and satisfies the Bellman optimality equation:

$$V^*(s) = \min_a \sum_{s'} P(s' \mid s,a) \, [\, C(s,a,s') + V^*(s') \,].$$

The expected value of an action combines its immediate transition cost with the values of all possible successor states. In a cooperative multi-agent problem, this calculation is performed over joint states because the outcome and future cost of one agent's action may depend on the positions and actions of the remaining agents.

2.2 Heuristic Value Initialization

A heuristic estimates the remaining cost from a state to a goal. Trial-based planners use this estimate to initialize states whose values have not yet been updated during planning. If h(s) denotes the heuristic value, an unseen state may initially be assigned V(s) = h(s).

A heuristic is admissible when it does not overestimate the optimal expected cost, so that h(s) <= V*(s). Such a heuristic is optimistic: it may omit delays or interactions that increase the true cost, but it must not predict a cost greater than the optimal one. In navigation problems, heuristic values are often based on remaining path distance and may also account for uncertainty in action execution.

For a multi-agent state, individual estimates can be combined into a joint lower bound.

2.3 Real-Time Dynamic Programming

Real-Time Dynamic Programming (RTDP) is a trial-based method for stochastic-shortest-path problems. Planning begins from an initial state and follows simulated trajectories through the environment. At each visited state, the planner performs a Bellman backup: it evaluates the expected immediate and future cost of every available action and replaces the stored state value with the lowest result. The action attaining this minimum is called the greedy action under the current value estimates, and one successor is then sampled from that action's transition distribution.

$$V(s) \leftarrow \min_a \sum_{s'} P(s' \mid s,a) \, [\, C(s,a,s') + V(s') \,].$$

A trial continues until it reaches a terminal state or another trial-ending condition, after which a new trial begins from the initial state. The collection of greedy actions assigned to the relevant states forms the current greedy policy. RTDP therefore updates only states encountered along this evolving policy rather than repeatedly sweeping over the entire state space.

The heuristic influences early trials by providing values for unvisited states. As planning proceeds, Bellman backups replace these initial estimates along trajectories that become relevant to the current policy.

2.4 LRTDP and Solved-State Labeling

Labeled Real-Time Dynamic Programming (LRTDP) augments trial-based planning with solved-state labels. These labels provide a convergence criterion for states relevant to the current greedy policy.

A solved state is different from a goal state. A goal state represents physical task completion. A solved state is a planning state whose value has become sufficiently consistent and whose relevant greedy successors have also been resolved.

Local consistency is measured through the Bellman residual:

$$\text{residual}(s) = \left| V(s) - \min_a \sum_{s'} P(s' \mid s,a) \, [\, C(s,a,s') + V(s') \,] \right|.$$

A small residual indicates that another Bellman backup would not substantially change the stored value. Solved-state labeling also examines states reachable with positive probability under the current greedy action. A state may be labeled solved when its residual is within tolerance and every relevant successor is a goal state, already solved, or part of the same locally consistent greedy-policy region.

When the initial state is labeled solved, the planner treats the policy relevant to that state as converged.

2.5 Operator Decomposition

A joint action contains one local action for every agent. If each of N agents has m available local actions, the number of complete joint actions is m^N. The cost of evaluating all combinations therefore grows exponentially with the team size. Operator decomposition addresses this joint-action branching by replacing a simultaneous choice with a sequence of local commitments.

Operator decomposition constructs the joint action through a sequence of local choices. An intermediate planning state stores the real joint state together with the partial action prefix selected so far. For three agents, the sequence may be represented as (s, empty), (s, (a1)), (s, (a1,a2)), and (s, (a1,a2,a3)).

Each intermediate decision compares at most m local continuations. The physical environment transition is not applied until the prefix contains one action for every agent. The sequential construction is therefore a computational serialization of a simultaneous joint action, not a turn-taking execution model. The agents still act simultaneously in the underlying MMDP, while the planning representation becomes deeper and locally less branching.

The decomposition introduces values for intermediate prefix states and requires several local selections to form one complete action. It reduces the joint-action bottleneck but does not reduce the number of possible physical joint states.

3. Problem Description

The problem consists of cooperative navigation on grid maps containing traversable cells and static obstacles. Each agent is assigned one start cell and one goal cell. The state of the system is an ordered tuple of agent positions, and every environment step requires one local action for each agent.

$$s = (p1, p2, ..., pN),\ a = (a1, a2, ..., aN).$$

Each agent can move Up, Down, Left, or Right, or select Stay. Consequently, a team of N agents has 5^N possible complete joint actions in every real state. The joint representation is necessary because movement conflicts and future costs depend on the configuration of the complete team.

Action execution is stochastic. An intended movement succeeds with probability 0.8 and results in no movement with probability 0.2. Slip outcomes are sampled independently for the agents before conflicts are resolved. An action directed outside the map or into an obstacle also leaves the agent in its current cell.

Two conflict types are prohibited. A vertex conflict occurs when two agents would occupy the same cell after a transition. An edge-swap conflict occurs when two agents would exchange positions during the same step. If a sampled joint outcome contains either conflict, the complete joint movement is rejected and the environment remains in the current state.

Agents that reach their assigned goals are frozen at those positions. A goal state is reached only when every agent occupies its assigned destination. The immediate cost of a real transition equals the number of agents that have not yet reached their goals:

$$C(s,a,s') = |\{ i : pi \mathrel{!}= gi \}|.$$

An unfinished agent contributes one unit of cost during every step before its arrival. The accumulated team cost therefore corresponds to the sum of individual arrival times. The objective is to minimize the expected cumulative cost until all tasks are completed.

The computational challenge has two components. The number of real joint states grows with combinations of agent positions, while the action set within each state grows as 5^N. Baseline RTDP evaluates the complete joint-action set directly. OD-RTDP changes only the representation of action selection, allowing the experiment to examine whether reducing local branching improves practical scalability.

4. Research Question and Hypotheses

The central research question is: How does operator decomposition affect planning time and memory consumption as the number of coordinated agents increases?

The comparison involves two competing effects. Direct joint-action evaluation grows exponentially with the team size, whereas OD-RTDP replaces this evaluation with a sequence of smaller local decisions. Although OD-RTDP introduces intermediate prefix states, it avoids explicitly evaluating and retaining the full set of complete joint-action alternatives at each decision point. The expected reduction in joint-action computation may therefore also reduce peak planning memory.

H1 - Planning time. As the number of agents increases, OD-RTDP is expected to become more time-efficient than Baseline RTDP because it avoids direct evaluation of the complete joint-action space.

H2 - Memory consumption. As the number of agents increases, OD-RTDP is expected to require less peak planning memory than Baseline RTDP because it avoids direct evaluation of the complete joint-action set and reduces the associated temporary computational structures. This expected saving is predicted to outweigh the additional storage required for intermediate action-prefix states.

5. Methodology

5.1 Environment and Transition Implementation

The environments are loaded from grid maps and scenario files containing fixed start-goal assignments. Each selected task is checked for valid free cells, individual reachability, and non-duplicated start and goal locations. A real state is stored as an ordered tuple so that each position remains associated with the corresponding agent and destination.

For a selected joint action, the transition model first generates the possible local outcomes caused by successful movement, slip, obstacles, or map boundaries. The local distributions are combined into joint outcomes. Conflict detection is then applied, and probabilities leading to the same successor state are aggregated.

Both planners use the same real-state transition model. Intermediate OD decisions do not move the agents, sample slip, resolve conflicts, or incur real transition cost.

5.2 Cost and Goal Handling

The task is implemented as an undiscounted SSP. Goal states contain the complete set of agents at their destinations, and agents remain frozen after arrival. The cost of each real transition is calculated from the current state as the number of unfinished agents.

No environment cost is charged while OD-RTDP constructs a partial prefix. A cost is applied only when the prefix is complete and the resulting joint action produces a real stochastic transition.

5.3 Shared Obstacle- and Slip-Aware Heuristic

Both planners use the same heuristic initialization. For each agent, a reverse breadth-first search (reverse BFS) begins at the goal and expands outward through the free cells. Running BFS from the goal, rather than separately from every possible agent position, computes the exact obstacle-aware shortest distance d_i(s) from every reachable cell to that goal in one traversal.

Let q = 1 - slip denote the probability that an intended movement succeeds. The heuristic value of a real joint state is:

$$h(s) = sum\_i\ di(s) / q.$$

Each term estimates the expected isolated cost required for an agent to complete its remaining shortest path under stochastic action execution. The joint heuristic sums these estimates across all unfinished agents.

For an intermediate OD state (s, alpha), the heuristic distinguishes between agents whose local actions have already been committed and agents whose actions remain undecided. Committed agents use a capped isolated estimate based on the current position and the selected action, while uncommitted agents use an optimistic local continuation. With an empty prefix, the OD heuristic is equal to the heuristic of the baseline at the same real state.

The implementation stores and initializes state values rather than maintaining a separate persistent Q-table. For every previously unseen state, V(s) is initialized with the shared heuristic h(s). During each Bellman backup, the action value Q(s,a) is computed on demand from the immediate transition cost and the current values of the possible successor states. The heuristic therefore initializes V directly and initializes the action-value estimates indirectly through the successor-state values used in this calculation.

5.4 Planning Procedure and LRTDP Stopping

Both planners use repeated trials beginning from the initial joint state. At each visited planning state, they apply the Bellman backup and greedy-action selection defined in Section 2.3, and then sample a successor. Baseline planning steps operate directly on real states. OD planning may pass through several prefix states before one complete real transition occurs.

Previously unseen planning states are initialized by the shared heuristic. Values are updated asynchronously along the trajectories produced by the current greedy policy.

The solved-state procedure tests Bellman consistency using an absolute tolerance of 10^-8 and a relative tolerance of 10^-6. A state passes the numerical residual test when |V_new - V_old| is no greater than 10^-8 + 10^-6 × max(1, |V_old|, |V_new|). The absolute component controls small numerical differences near zero, while the relative component scales the accepted difference to the magnitude of the value. The procedure then examines all successors that have positive probability under the greedy action. A state is labeled solved only when the relevant greedy-policy region satisfies these consistency conditions.

When planning terminates because the initial state is labeled solved, the greedy-policy region relevant to that state has passed the LRTDP Bellman-consistency test under the admissible heuristic and the configured numerical tolerances. Because the tolerances are nonzero, this result is interpreted as approximate numerical convergence rather than an exact symbolic proof that the stored values equal the mathematical optimum. When planning terminates because the 60-second planning limit or the 75-second external watchdog is reached, no optimality guarantee is made for the returned policy.

5.5 Policy Evaluation Procedure

After planning, the final value estimates are frozen and the resulting greedy policy is evaluated through stochastic rollouts. A rollout is one simulated execution episode that starts from the initial state and follows the resulting policy while stochastic action outcomes are sampled. At each evaluation state, the policy selects the action with the lowest estimated expected cost under the computed values. No additional Bellman backups are performed during evaluation. Every rollout uses the same slip, conflict, cost, and goal rules as the planning model.

6. Experimental Setup

6.1 Maps and Conditions

The experiment uses three MovingAI-style octile grid environments that differ in both scale and internal structure. The open-grid map (empty-8-8) is an 8×8 obstacle-free environment and therefore serves as the simplest condition, with unrestricted movement except for stochastic action failure and agent interactions. The warehouse map (warehouse-10-20-10-2-1) is a 161×63 environment with long structured aisles, storage blocks, and restricted lateral movement, creating repeated coordination bottlenecks. The room map (room-64-64-16) is a 64×64 environment composed of multiple rooms, walls, and narrow connecting passages, making coordination more difficult because agents must navigate through confined regions and shared chokepoints.

Each map is evaluated with teams of one to six agents using Baseline RTDP and OD-RTDP, producing 36 planned conditions in total. In every condition, the paired planners receive the same map, task assignment, MMDP parameters, and random seed so that the comparison isolates the effect of the planning method rather than differences in the underlying task instance. Figure 1 shows the three experimental maps.

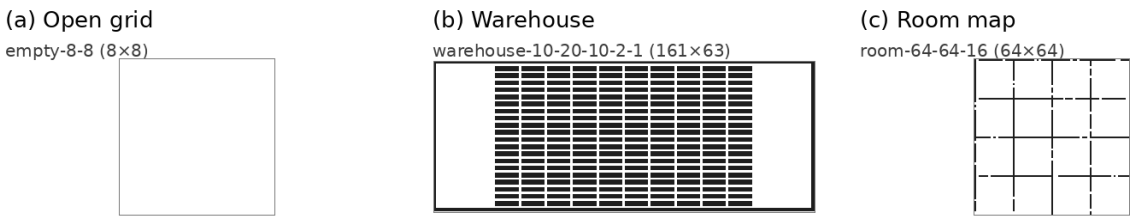

Figure 1. Experimental maps used in the study. (a) Open 8×8 grid with no internal obstacles. (b) Warehouse 161×63 map with structured aisles and restricted movement corridors. (c) Room 64×64 map with multiple rooms, walls, and narrow passages.

6.2 Execution Protocol and Resource Controls

Each condition is executed in a separate process so that memory retained by one run does not affect the next measurement. Planning continues until the initial state is labeled solved or the 60-second planning budget is exhausted.

An external watchdog is a separate supervising process that runs outside the planner, monitors the complete experimental condition, and forcibly terminates it if no result is returned within 75 seconds. Peak additional memory is measured relative to the beginning of the planning phase.

6.3 Evaluation Protocol

Five evaluation episodes are scheduled for every condition. The evaluation stage has an eight-second wall-clock limit, and each episode is successful only when all agents reach their assigned goals within the map-specific step limit. The stage stops scheduling additional episodes when its time limit is reached.

The results use a fixed denominator of five scheduled episodes. Episodes that are not completed within the eight-second evaluation budget are counted as unsuccessful under this budgeted evaluation measure. A condition terminated by the external watchdog is also reported as 0/5, with the timeout identified separately.

6.4 Outcome Measures

The reported outcomes are defined separately from the evaluation procedure. Execution status distinguishes conditions that return normally from conditions stopped by the watchdog defined in Section 6.2. Among normally returned conditions, the planning stop reason distinguishes convergence of the initial state from exhaustion of the 60-second planning budget.

Planning time is interpreted as time to convergence only when the initial state is labeled solved. A value near 60 seconds in a budget-limited condition indicates that the available planning time was consumed; it is not a measured time to solution.

Peak additional memory is the largest increase in process memory observed during the planning phase. Policy performance is reported as successful evaluation episodes out of five scheduled episodes, where an episode succeeds only if all agents reach their goals within the evaluation limits. No single measure is treated as a complete description of performance: planning convergence, memory, condition completion, and policy evaluation are interpreted together.

7. Results

7.1 Completion Overview

Of the 36 planned conditions, 16 terminate after the initial state is labeled solved, 17 consume the complete 60-second planning budget, and three are terminated by the external watchdog without returning complete internal planning and evaluation measurements.

All open-grid OD-RTDP conditions reach the solved-state criterion. The baseline reaches the criterion for one to four agents and consumes the planning budget for five and six agents. On the warehouse map, both planners converge for one and two agents and reach the planning limit for three to five agents; at six agents, OD-RTDP returns after reaching the planning limit while the baseline is terminated by the watchdog. On the room map, both planners converge only for one agent, both reach the planning limit for two to five agents, and both six-agent conditions are terminated by the watchdog.

7.2 Planning Time

Table 1 reports all six open-grid conditions, where the crossover between the planners can be measured directly. Entries marked as solved represent time to convergence. Entries marked as time limit indicate that the planner consumed the full budget without reaching the solved-state criterion.

Agents	Baseline RTDP	OD-RTDP	Comparison
1	0.0012 s - solved	0.0021 s - solved	Baseline 1.8x faster
2	0.0061 s - solved	0.0085 s - solved	Baseline 1.4x faster
3	0.2000 s - solved	0.0491 s - solved	OD 4.1x faster
4	7.9701 s - solved	0.3783 s - solved	OD 21.1x faster

Agents	Baseline RTDP	OD-RTDP	Comparison
5	60.0004 s - time limit	2.8722 s - solved	Only OD converged
6	60.0194 s - time limit	16.2720 s - solved	Only OD converged

Table 1. Planning outcomes on the open-grid map. A time-limit entry is not a time-to-solution measurement.

Baseline RTDP is faster for one and two agents, when direct joint-action enumeration remains inexpensive. OD-RTDP becomes 4.1 times faster at three agents and 21.1 times faster at four agents. In the five- and six-agent conditions, OD-RTDP reaches the solved-state criterion in 2.87 and 16.27 seconds, respectively, while the baseline consumes the complete 60-second planning budget without converging.

On the warehouse map, Baseline RTDP converges in 0.0071 and 3.1424 seconds for one and two agents, compared with 0.0322 and 4.4154 seconds for OD-RTDP. Neither planner converges from three agents onward; the six-agent baseline condition is terminated by the watchdog, while OD-RTDP returns after reaching the planning limit. On the room map, only the single-agent conditions converge, in 0.0263 seconds for the baseline and 0.0747 seconds for OD-RTDP. Both planners reach the planning limit from two to five agents, and both six-agent conditions time out.

7.3 Peak Planning Memory

Figures 2a-2c report peak additional planning memory separately for each map. Hatched bars identify conditions that reached the 60-second planning time limit; the hatching does not indicate a memory limit. Timeout markers represent conditions for which no memory measurement was returned.

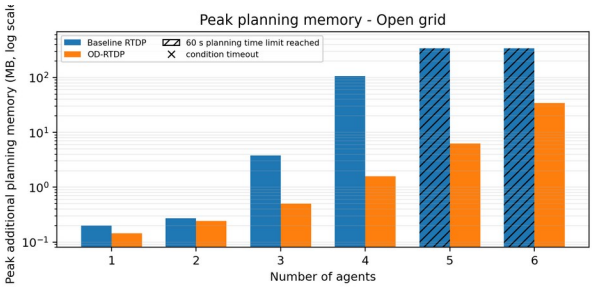

Figure 2a. Peak additional planning memory on the open-grid map.

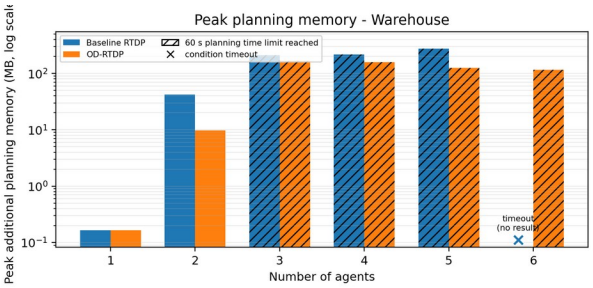

Figure 2b. Peak additional planning memory on the warehouse map. The timeout marker denotes the condition with no returned measurement.

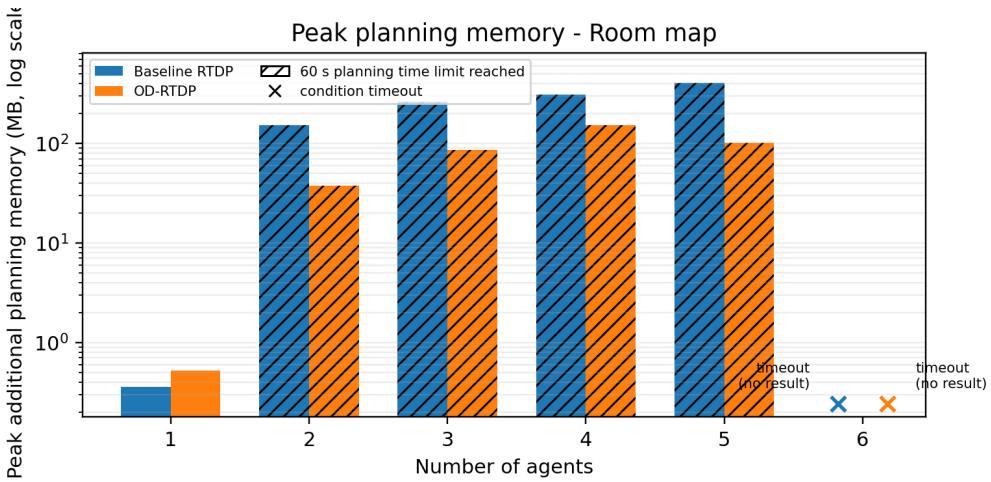

Figure 2c. Peak additional planning memory on the room map. Timeout markers denote conditions with no returned measurements.

The open-grid results strongly support the memory hypothesis. OD-RTDP uses less peak planning memory at every agent count, although the differences at one and two agents are small. At four agents, the baseline uses 106.68 MB compared with 1.59 MB for OD-RTDP; at five agents, the corresponding values are 343.69 MB and 6.31 MB; and at six agents they are 341.95 MB and 34.45 MB.

On the warehouse map, the one-agent measurements are equal at 0.16 MB, while OD-RTDP uses less memory in every returned multi-agent pair. For example, the two-agent values are 42.46 MB for the baseline and 9.73 MB for OD-RTDP, while the five-agent values are 275.58 MB and 124.13 MB. On the room map, OD-RTDP uses slightly more memory at one agent but substantially less from two to five agents; at five agents, the values are 406.84 MB for the baseline and 101.06 MB for OD-RTDP. Memory cannot be compared directly in watchdog-terminated conditions because those conditions return no measurement.

7.4 Budgeted Policy Evaluation

Table 2 reports successful evaluation episodes out of five scheduled episodes. The fixed denominator treats episodes that could not be completed within the condition budget as unsuccessful. The table therefore represents a budgeted end-to-end outcome rather than an unconstrained estimate of policy success probability.

Agents	Open B	Open OD	Warehouse B	Warehouse OD	Room B	Room OD
1	5/5	5/5	5/5	5/5	5/5	5/5
2	5/5	5/5	5/5	5/5	5/5	5/5
3	5/5	5/5	5/5	5/5	5/5	4/5
4	5/5	5/5	5/5	5/5	2/5	1/5
5	5/5	5/5	4/5	5/5	0/5	0/5
6	4/5	5/5	0/5 (timeout)	3/5	0/5 (timeout)	0/5 (timeout)

Table 2. Successful evaluation episodes out of five scheduled episodes. B denotes Baseline RTDP; OD denotes OD-RTDP. Uncompleted episodes are counted as unsuccessful under the budgeted measure.

On the open grid, both planners achieve 5/5 from one to five agents. At six agents, OD-RTDP achieves 5/5, while the baseline completes four successful episodes and receives 4/5 under the fixed-denominator measure. On the warehouse map, both planners achieve 5/5 from one to four agents. At five agents, the baseline obtains 4/5 and OD-RTDP obtains 5/5. At six agents, OD-RTDP returns 3/5, while the baseline condition is terminated by the watchdog and is reported as 0/5.

The room-map results weaken as the team grows. Both planners achieve 5/5 at one and two agents. At three agents, the baseline obtains 5/5 and OD-RTDP obtains 4/5. At four agents, the baseline obtains 2/5 and OD-RTDP obtains 1/5. At five agents, both planners return 0/5, and at six agents both conditions are terminated by the watchdog.

The budgeted evaluation also distinguishes policy success from condition completion. At six agents on the warehouse map, OD-RTDP completes three successful episodes and returns 3/5, while the baseline is terminated by the watchdog. At five agents on the room map, both planners return valid 0/5 results: the baseline completes one failed episode and OD-RTDP completes three failed episodes before the evaluation budget expires. In the hardest six-agent room condition, neither planner completes the full experimental pipeline within 75 seconds.

8. Discussion

8.1 Interpretation of the Hypotheses

H1 predicted that OD-RTDP would become more time-efficient than Baseline RTDP as the number of agents increased. The open-grid results support this hypothesis in a scale-dependent form. Baseline RTDP is faster for one and two agents, where the complete joint-action set remains small and the additional OD depth is pure overhead. From three agents onward, the reduction in local branching outweighs this overhead; at five and six agents, only OD-RTDP reaches the solved-state criterion within the common planning budget.

The structured-map results provide more limited support for H1 because many larger conditions consume the same 60-second planning budget or are censored by the external watchdog. They therefore do not establish that OD-RTDP converges faster in every environment. Instead, they show that operator decomposition is most useful when complete joint-action enumeration is the dominant computational bottleneck, while joint-state exploration, congestion, and stochastic interaction may still determine the overall difficulty.

H2 predicted that OD-RTDP would require less peak planning memory as team size increased. The results support this hypothesis more consistently: OD-RTDP uses less peak planning memory in every returned multi-agent comparison. This indicates that, at the tested scales, the savings from avoiding direct complete-action evaluation outweigh the storage required for intermediate prefix states. The small single-agent exceptions also show that the memory advantage is scale dependent rather than inherent at every problem size.

9. Limitations

The experiment uses one fixed seed for planning and evaluation and one task selection for each map. The stochastic inputs and paired conditions are reproducible, but wall-clock time and process-memory measurements may vary with hardware, operating-system scheduling, and concurrent system load. The single-seed design also does not estimate variability across alternative stochastic sequences or start-goal assignments.

The budgeted policy measure uses five scheduled episodes, and episodes that are not completed within the available time are counted as unsuccessful. This convention supports a consistent end-to-end comparison, but it combines policy behavior with the computational ability to finish evaluation. It should not be interpreted as an unconstrained success probability.

Three conditions are terminated by the external watchdog without returning complete internal measurements: the six-agent warehouse baseline and both six-agent room conditions. These outcomes remain part of the analysis but prevent complete paired comparisons of planning memory and policy evaluation in those conditions.

The benchmark includes three map structures but does not cover the full range of possible multi-agent environments. The model also assumes centralized access to the complete joint state and a known transition model; decentralized observation, communication, and coordination among independently planning agents are not examined. The OD implementation uses a fixed agent order, and the shared heuristic does not represent interaction between agents.

10. Future Work

Future experiments should repeat each condition across multiple planning seeds and task selections, allowing paired variation and confidence intervals to be reported. A larger evaluation budget would also permit a less compressed assessment of policy behavior in difficult conditions.

Further algorithmic work could examine dynamic agent ordering, heuristics that incorporate predicted interaction, and factored representations of states, transitions, or value functions. Such structure could exploit weak dependencies between agents or map regions and address part of the joint-state bottleneck that remains after operator decomposition reduces joint-action branching. Broader benchmark suites would help determine whether the observed crossover persists under different forms of congestion and coordination.

11. Development Process and Design Rationale

11.1 Evolution of the Research Focus

The project began as a direct test of whether operator decomposition could reduce the cost of complete joint-action evaluation. Development showed that lower branching does not automatically produce a faster planner: OD also introduces prefix states, additional decision depth, and heuristic requirements for partial actions. The final research question was therefore framed around the computational crossover between these effects. This made the Baseline advantage for small teams an expected part of the analysis, while planning time and peak memory became the main indicators of when decomposition begins to justify its overhead.

11.2 Modeling Choices and Experimental Fairness

The stochastic movement model requires a policy rather than a fixed path because an intended action may become Stay and lead to an unplanned joint state. Vertex and edge-swap conflicts reject the sampled joint movement instead of applying a hand-tuned penalty, so conflicts remain costly through delay while the transition model stays identical for both planners. Freezing completed agents and charging one unit for each unfinished agent makes cumulative cost equal to the sum of individual arrival times. To isolate operator decomposition, both algorithms share the same environment, transition rules, cost, heuristic source, RTDP/LRTDP engine, seed, resource limits, and evaluation procedure; only the representation of action selection differs.

11.3 Heuristic and Stopping Decisions

Reverse BFS was chosen because it computes exact obstacle-aware distances and provides stronger guidance than Manhattan distance on maps with walls, aisles, and narrow passages. Dividing by the movement-success probability estimates the isolated effort required under slip, while omitting inter-agent delays keeps the initialization optimistic. The OD prefix heuristic was designed to remain compatible with the Baseline value at an empty prefix. During development, simple value stability was replaced by an LRTDP-style solved-state test over the relevant greedy-policy region. A solved result therefore indicates approximate Bellman consistency under the configured tolerances; a result produced by the time limit or watchdog carries no optimality guarantee.

11.4 Final Experimental Scope and Submission

The final matrix uses three structurally different maps and teams of one to six agents, which is sufficient to expose both OD overhead at small scales and its later crossover. Conditions run serially in fresh processes so that memory measurements remain isolated, while explicit planning, evaluation, and watchdog limits keep the full matrix executable. Development-only diagnostics and unused configuration paths were removed before submission, leaving a compact result schema that matches the reported tables and figures. The resulting workflow connects the implemented algorithms, experimental controls, analysis, and conclusions without retaining instrumentation that is irrelevant to the final study.

Taken together, these decisions turned the initial prototype into a controlled empirical comparison that identifies both the practical benefit of operator decomposition and the limits that remain when joint-state complexity dominates.

12. Conclusion

The central conclusion of this study is that operator decomposition provides a more scalable planning representation as the number of coordinated agents increases. Baseline RTDP evaluates a complete joint-action set whose size grows as 5^N, so the cost of each backup rises rapidly with team size. OD-RTDP replaces this direct enumeration with a sequence of locally bounded choices. The experiments do not prove a new formal asymptotic bound for the complete planner, but they show that planning time and peak memory grow substantially more slowly in practice under OD-RTDP across the tested range.

Baseline RTDP remains advantageous for small teams. Its representation is direct, it avoids prefix states and additional decision depth, and it is faster when the joint-action set is still manageable. It can also match or exceed OD-RTDP in policy evaluation on some structured-map conditions. Its disadvantage is poor scalability: as agents are added, every backup must evaluate an exponentially growing set of action combinations, causing time and temporary memory requirements to increase sharply.

OD-RTDP has the complementary profile. It introduces prefix-state storage, greater decision depth, and a fixed agent order, and it does not guarantee better policy behavior on every map. In return, it greatly reduces the joint-action bottleneck, provides substantially more memory headroom, and extends the range of team sizes that can be handled within fixed resources. The practical choice is therefore scale dependent: Baseline RTDP is preferable for small, low-branching instances, whereas OD-RTDP is the stronger computational foundation for larger cooperative teams. Operator decomposition delays the point at which joint-action evaluation becomes impractical, although it does not remove the remaining difficulty of joint-state growth.

Project repository: https://github.com/t-rays/MMDP-OD-RTDP-PROJECT